

Table Design – Compression in Amazon Redshift

To Begin with the Lab

Summary of the Lab

In this lab, compression encoding in **Amazon Redshift** was analyzed and optimized to improve storage efficiency and query performance. The existing compression settings of the usersauto table were examined using Redshift system catalog views, and compression recommendations were generated using the ANALYZE COMPRESSION command. Based on the recommendations, a new table was created with custom ZSTD compression applied to selected columns. Data from the original users table was loaded into the new table, and the applied encodings were verified. Finally, compression analysis was executed again to confirm that optimized columns were already using the recommended encodings while other columns continued to receive additional recommendations.

Understanding Compression in Amazon Redshift

Compression reduces the amount of storage required by a table while improving query performance. Since **Amazon Redshift** reads less data from disk, queries often execute faster and consume fewer resources. Redshift automatically applies compression encodings to many columns, but administrators can also define custom encodings when creating or modifying tables.

Example:

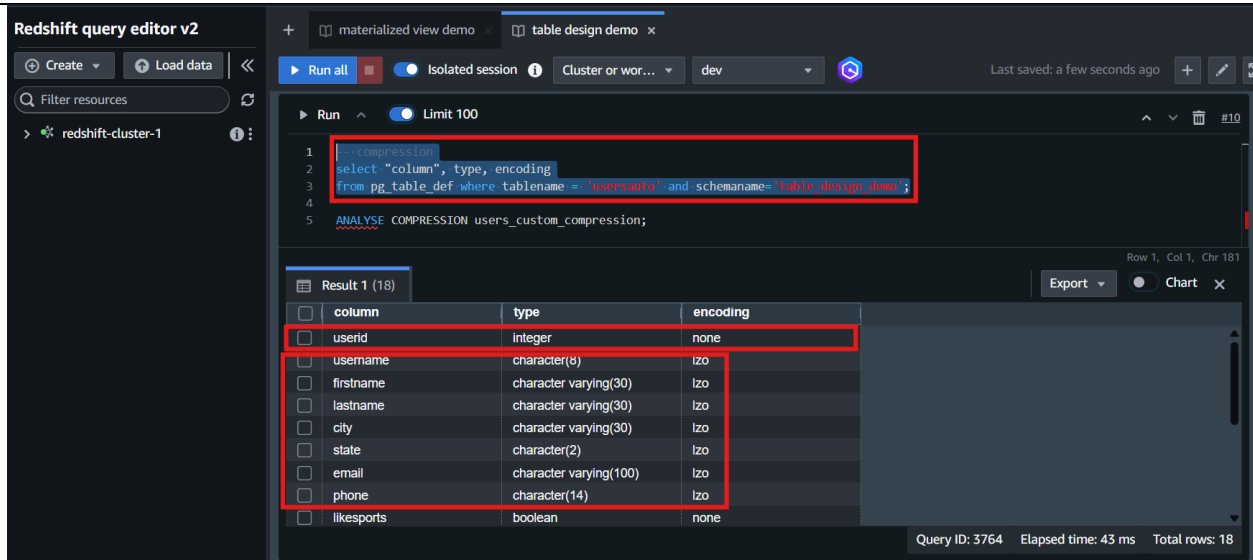
Consider a customer table containing thousands of repeated first names, last names, cities, and email domains. Instead of storing every value in its original form, compression algorithms store the data more efficiently. If a column requires 100 MB without compression, a suitable encoding such as ZSTD may reduce storage significantly while still returning the same query results. This reduction means fewer disk blocks are scanned, leading to faster query execution.

Lab Steps

- Sign in to the AWS Management Console
- Navigate to **Amazon Redshift** → open **Query Editor v2**
- Review the existing compression encodings of the usersauto table

```
SELECT tablename,  
       "column",  
       encoding
```

```
FROM pg_table_def
WHERE tablename = 'usersauto';
```



Redshift query editor v2

Run all Isolated session Cluster or wor... dev Last saved: a few seconds ago

Filter resources

redshift-cluster-1

Run Limit 100

```
1 select 'column', type, encoding
2 from pg_table_def where tablename = 'usersauto' and schemaname = 'table design demo';
3
4 ANALYZE COMPRESSION users_custom_compression;
```

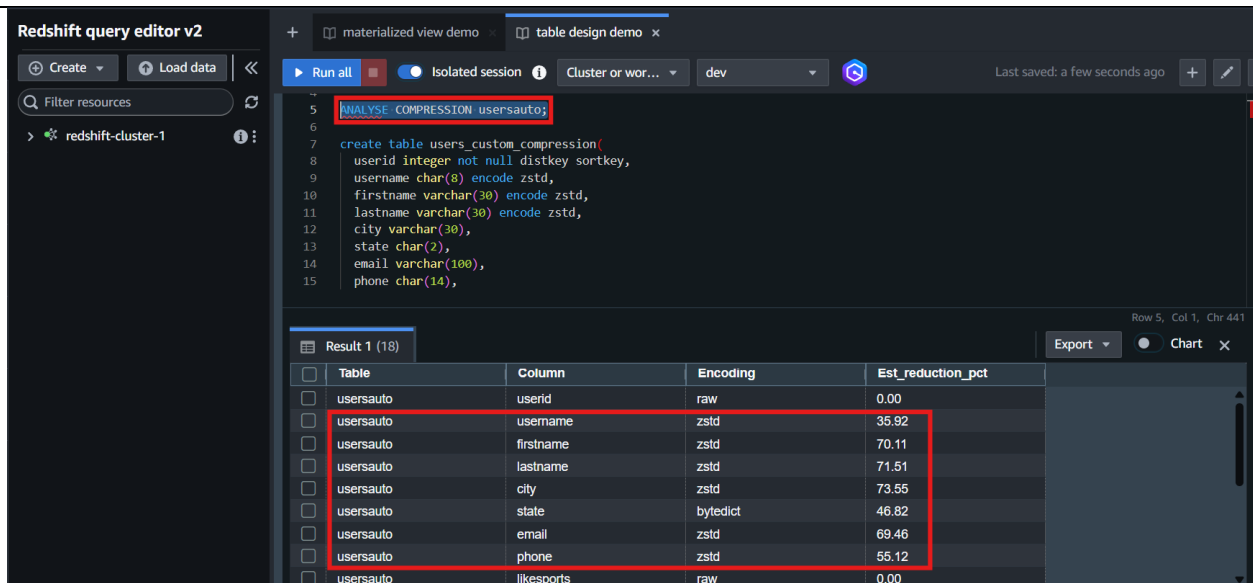
Result 1 (18)

column	type	encoding
userid	integer	none
username	character(8)	lzo
firstname	character varying(30)	lzo
lastname	character varying(30)	lzo
city	character varying(30)	lzo
state	character(2)	lzo
email	character varying(100)	lzo
phone	character(14)	lzo
likesports	boolean	none

Query ID: 3764 Elapsed time: 43 ms Total rows: 18

- Verify that userid has no compression encoding
- Understand that userid is configured as a distribution key and sort key
- Understand that **Amazon Redshift** does not compress distribution keys and sort keys
- Verify that character columns such as username, firstname, lastname, and phone use LZO encoding by default
- Analyze compression recommendations

```
ANALYZE COMPRESSION usersauto;
```



Redshift query editor v2

Run all Isolated session Cluster or wor... dev Last saved: a few seconds ago

Filter resources

redshift-cluster-1

Run Limit 100

```
5 ANALYZE COMPRESSION usersauto;
```

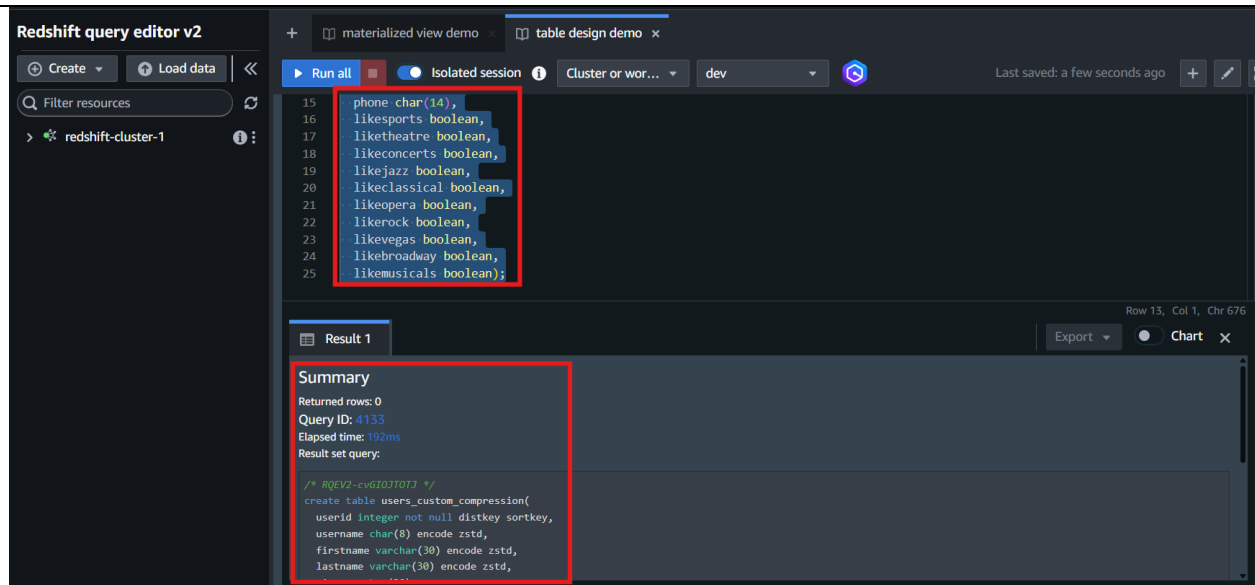
Result 1 (18)

Table	Column	Encoding	Est_reduction_pct
usersauto	userid	raw	0.00
usersauto	username	zstd	35.92
usersauto	firstname	zstd	70.11
usersauto	lastname	zstd	71.51
usersauto	city	zstd	73.55
usersauto	state	bytedict	46.82
usersauto	email	zstd	69.46
usersauto	phone	zstd	55.12
usersauto	likesports	raw	0.00

- Review the recommended encodings generated by **Amazon Redshift**
- Review the estimated storage reduction percentages
- Verify that username, firstname, and lastname are recommended to use ZSTD encoding

- Understand that ANALYZE COMPRESSION only generates recommendations and does not modify the table
- Create a table with custom compression encodings

```
CREATE TABLE users_custom_encoding (
  userid INTEGER,
  username VARCHAR ENCODE zstd,
  firstname VARCHAR ENCODE zstd,
  lastname VARCHAR ENCODE zstd,
  city VARCHAR
);
```



- Click **Run**
- Verify that the table is created successfully
- Insert data into the new table

```
INSERT INTO users_custom_encoding
SELECT userid,
       username,
       firstname,
       lastname,
       city
FROM public.users;
```

The screenshot shows the Redshift query editor v2 interface. The query editor has tabs for 'materialized view demo' and 'table design demo'. The 'table design demo' tab is active, showing a SQL query. The query is as follows:

```

1  -- inserting records from users
2  insert into users_custom_compression
3  select * from users;
4
5  select "column", type, encoding
6  from pg_table_def where tablename = 'users_custom_compression' and schemaname='table_design_demo';
7
8  ANALYSE COMPRESSION users_custom_compression;

```

The query is highlighted with a red box. Below the query editor, the 'Result 1' tab is selected, showing a summary of the query results:

Summary
Affected rows: 49990
Query ID: 4200
Elapsed time: 1.3s
Result set query:

```

/* RQEVO-ATaBuTPdy6 */
-- inserting records from users
insert into users_custom_compression
select * from users

```

- Click **Run**
- Verify that data is copied successfully from the users table
- Verify the compression encodings of the new table

```

SELECT tablename,
       "column",
       encoding
FROM pg_table_def
WHERE tablename = 'users_custom_encoding';

```

The screenshot shows the Redshift query editor v2 interface. The query editor has tabs for 'materialized view demo' and 'table design demo'. The 'table design demo' tab is active, showing a SQL query. The query is as follows:

```

1  -- inserting records from users
2  insert into users_custom_compression
3  select * from users;
4
5  select "column", type, encoding
6  from pg_table_def where tablename = 'users_custom_compression' and schemaname='table_design_demo';
7
8  ANALYSE COMPRESSION users_custom_compression;

```

The query is highlighted with a red box. Below the query editor, the 'Result 1 (18)' tab is selected, showing a table of results:

column	type	encoding
userid	integer	none
username	character(8)	zstd
firstname	character varying(30)	zstd
lastname	character varying(30)	zstd
city	character varying(30)	lzo
state	character(2)	lzo
email	character varying(100)	lzo
phone	character(14)	lzo
likesports	boolean	none

The table is highlighted with a red box. At the bottom right, the status bar shows: Query ID: 4200 Elapsed time: 94 ms Total rows: 18

- Verify that username uses ZSTD encoding
- Verify that firstname uses ZSTD encoding
- Verify that lastname uses ZSTD encoding
- Verify that other character columns continue using the default LZO encoding
- Run compression analysis again

ANALYZE COMPRESSION users_custom_encoding;
--

- Review the new recommendations
- Verify that username, firstname, and lastname are already using the recommended ZSTD encoding
- Verify that additional recommendations are generated only for columns that still use default encodings
- Verify that custom compression is applied successfully
- Verify that storage optimization recommendations are implemented correctly
- Verify that compression helps reduce storage usage, improve query performance, and reduce scan costs in **Amazon Redshift**